

The Circular Buffer: A Queue Backed by One Array

Programming and Data Structures — Week 3, Lecture 3

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain how a circular buffer avoids the $O(n)$ dequeue trap from the previous lecture without abandoning contiguous arrays altogether; compute the `front` and `back` positions of a circular buffer using modular arithmetic, by hand, for a given sequence of operations; implement a fixed-capacity circular queue entirely from scratch; and correctly distinguish a full buffer from an empty one, a classic and easy-to-get-wrong edge case in this structure.

Outline

Today covers a running-track analogy for circular structure; why `collections.deque` from the previous lecture is not the only way to solve the array-backed-queue problem; the circular buffer idea of wrapping indices around instead of shifting elements; the modular arithmetic that computes `front` and `back` positions, worked through fully with numeric examples; the classic full-versus-empty ambiguity this structure creates and how to resolve it correctly; a from-scratch implementation of `CircularQueue`; the MTech-track material on where fixed-capacity circular buffers appear in real systems; an in-class quiz; and a summary.

The running-track analogy

Picture a runner on a standard 400-meter running track: the track has a fixed physical length, yet the runner never runs off the end of it — reaching what would be the “end” of a straight line just means the next step continues from the start again. A circular buffer is a fixed-size array that behaves exactly like this track: indices increase as usual, but upon reaching the last slot, the next index wraps back around to slot 0, rather than requiring a new, larger block of memory the way Week 2’s dynamic array did.

Why not just always use deque?

`collections.deque` from the previous lecture is an excellent general-purpose solution, genuinely $O(1)$ at both ends. But it allocates memory dynamically in blocks as elements are added, with no hard, pre-declared upper bound on memory use. Some systems cannot tolerate this: an embedded microcontroller with a fixed amount of RAM, an audio processing pipeline with a strict real-time buffer size, or a network driver's fixed-size packet buffer all need a data structure whose memory footprint is known exactly, in advance, and never grows. A circular buffer built over one fixed-size array provides precisely this guarantee, at the cost of a hard capacity limit.

The circular buffer idea

```
Array (capacity 5): [ _, _, _, _, _ ]  
                   0  1  2  3  4
```

```
enqueue(A): front=0, back=0 → [A, _, _, _, _], back moves to 1
```

```
enqueue(B): [A, B, _, _, _], back moves to 2
```

```
enqueue(C): [A, B, C, _, _], back moves to 3
```

```
dequeue(): removes A, front moves to 1 → logical queue is now [B, C]  
          (slot 0 is not shifted - it is simply no longer "in" the queue)
```

The key idea: rather than physically shifting elements when the front is removed, as the naive queue from Lecture 2 did, a circular buffer simply moves a `front` marker forward, leaving the vacated slot's memory untouched until it is eventually overwritten by a future `enqueue`. Because no element is ever physically moved, both `enqueue` and `dequeue` remain $O(1)$ with no hidden per-call shifting cost — the entire $O(n)$ trap from Lecture 2 is avoided structurally, not merely worked around.

Wrapping around: the key trick

```
back = (back + 1) % capacity
```

The single line of code that makes an ordinary array behave like a loop is the modulo operation: `back = (back + 1) % capacity`. For a capacity-5 buffer, this maps the sequence of positions 0, 1, 2, 3, 4, 5, 6, ... to 0, 1, 2, 3, 4, 0, 1, ... — index 5, which would be out of bounds for a straight array, wraps back to index 0 instead. This one operation is the complete mechanism that turns a straight, fixed-size array into a conceptually circular one.

Worked example — tracing back through a full wrap

Tracing back through 7 enqueues on a capacity-5 buffer, starting at `back = 0`: the first four enqueues advance `back` normally to 1, 2, 3, 4. The fifth enqueue computes $(4 + 1) \bmod 5 = 0$, wrapping `back` back to position 0 — the buffer has looped around exactly once. The sixth and seventh enqueues continue normally from there, to 1 and then 2. This table is worth reproducing by hand for a different capacity as practice before the lab.

The full-vs-empty ambiguity

A subtle and classic problem emerges once wrapping is introduced: both a completely empty buffer and a completely full buffer can end up with `front == back`. In the empty case, nothing has ever been added, so both markers start at the same position. In the full case, `back` has wrapped all the way around and caught back up to `front`. If the implementation tries to distinguish “empty” from “full” by comparing `front` and `back` alone, it cannot — both conditions look identical using only those two values, which is a well-known source of real bugs in circular buffer implementations.

Resolving the ambiguity: track count explicitly

```
class CircularQueue:
    def __init__(self, capacity):
        self.capacity = capacity
        self._data = [None] * capacity
        self.front = 0
        self.back = 0
        self.count = 0          # the tie-breaker: explicit element count

    def is_empty(self):
        return self.count == 0

    def is_full(self):
        return self.count == self.capacity
```

The cleanest fix is to stop relying on `front` and `back` alone and instead track the element count explicitly as a third piece of state. `is_empty()` and `is_full()` both become trivial, unambiguous checks against `count`, completely sidestepping the comparison problem. This is a general lesson worth internalizing: when two distinct states produce the same values for the variables you are checking, the fix is usually to track one more piece of state directly, rather than trying to cleverly re-derive it from existing variables.

The full enqueue and dequeue

```
def enqueue(self, x):
    if self.is_full():
        raise OverflowError("circular queue is full")
    self._data[self.back] = x
    self.back = (self.back + 1) % self.capacity
    self.count += 1

def dequeue(self):
    if self.is_empty():
        raise IndexError("dequeue from an empty queue")
    x = self._data[self.front]
    self.front = (self.front + 1) % self.capacity
    self.count -= 1
    return x
```

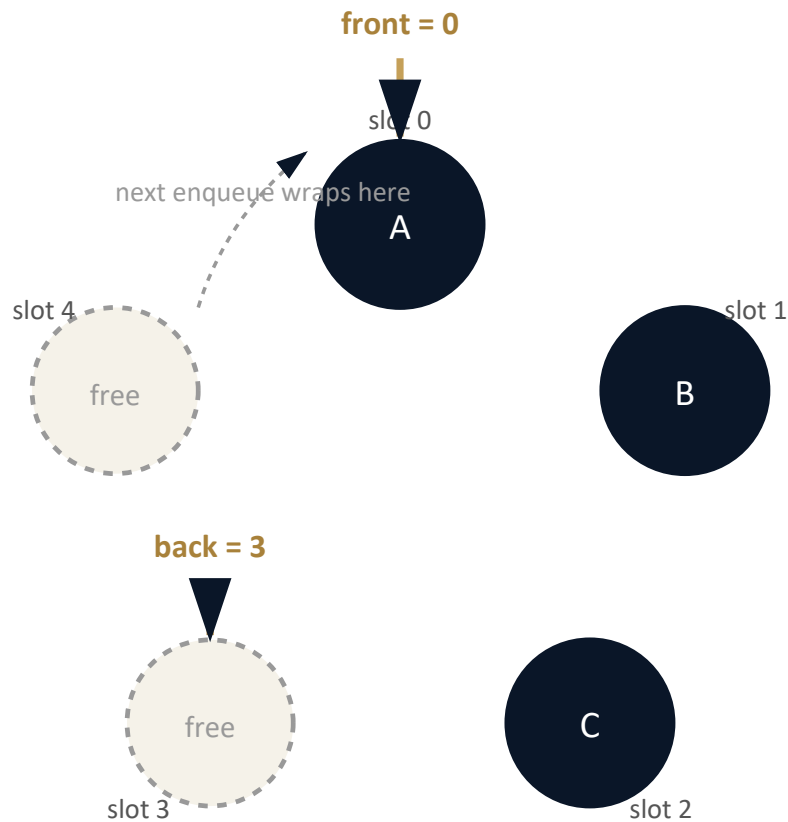
Both operations follow the same three-step pattern: check the boundary condition first (`is_full()` for enqueue, `is_empty()` for dequeue), perform the actual read or write at the current marker position, then advance that marker using the modulo wrap-around and update `count`. Every step here is $O(1)$ — no loop, no shifting — regardless of how many elements are currently stored or how many times the buffer has already wrapped around.

Full worked trace — fill, drain, wrap, refill

This full trace demonstrates the wrap-around in action on a small, capacity-3 buffer. After three enqueues the buffer is full and `back` has already wrapped to 0. The dequeue removes A and advances `front` to 1, dropping `count` to 2, which correctly un-marks the buffer as full. The final enqueue writes D into slot 0 — physically overwriting where A used to sit, which is now safe because A has already been logically removed — and `back` advances to 1, with `count` back at 3 (full again). The logical queue content at the end, read from `front` around to `back`, is D, B, C in that FIFO order, correctly reflecting that A was removed and D was the most recent addition.

The picture: the buffer as a loop

Capacity-5 circular buffer, 3 elements stored



Elements A, B, C occupy slots 0-2; slots 3-4 are free. The next enqueue writes at slot 3 (back)

Drawing the buffer as an actual ring, rather than a straight line, is often the representation that makes the wrap-around click for students seeing this structure for the first time — the array is straight in memory, but the *logical* structure it implements is genuinely circular.

MTech addition — where circular buffers appear in real systems

Circular buffers appear throughout systems programming precisely because of the fixed-memory guarantee this lecture has emphasized. Audio processing pipelines use ring buffers to hold a fixed recent window of sample data, continuously overwritten as new audio arrives. Operating system network drivers use packet ring buffers with a fixed, pre-allocated memory cost, avoiding the unpredictable allocation behavior of a dynamically growing structure under high network load. Logging systems often keep a “last N lines” ring buffer for diagnostics, deliberately overwriting the oldest entry once full rather than growing without bound — a deliberate design trade-off between completeness and guaranteed, fixed memory use.

MTech addition — overwrite-oldest vs. reject-new

This lecture’s `CircularQueue` implementation rejects new elements once full, raising `OverflowError`. A logging ring buffer typically makes the opposite choice: once full, the oldest entry is silently overwritten to make room for the newest one, since for diagnostic logging, losing an old line is usually preferable to losing the ability to record what is happening right now. Both policies use the identical underlying circular structure — the choice between them is a design decision about which kind of data loss an application can tolerate, not a difference in the data structure itself.

Reconnecting to Week 2’s amortized analysis

It is worth pausing to connect this lecture back to Week 2’s central lesson. A dynamic array’s doubling strategy trades some wasted memory for avoiding an $O(n^2)$ time cost under repeated appends. A circular buffer makes a different trade entirely: it refuses to grow at all, accepting a hard capacity limit in exchange for a memory footprint that is fixed and known exactly in advance. Both decisions follow the same underlying discipline — identify precisely which cost your application cannot afford (unpredictable time, in one case; unpredictable memory, in the other), and design the data structure specifically around refusing to pay that cost, accepting a different trade-off instead.

Exercise

As an exercise, implement an `enqueue_overwrite(x)` method that, when the buffer is full, overwrites the oldest element instead of raising `OverflowError`. Trace it by hand on a capacity-3 buffer already holding `[A, B, C]` (full), calling `enqueue_overwrite(D)`. Work out what should happen to `front` as a result, and identify explicitly which element is now permanently lost.

A second worked trace — a full lifecycle with two wraps

A capacity-2 buffer wraps quickly, which makes it a useful stress test for this structure. Tracing six alternating operations shows `back` wrapping from 1 to 0 after the third operation and `front` wrapping identically two steps later — both markers independently cycling through the same modulo-2 sequence, 0, 1, 0, 1, ..., exactly as the formula $(x + 1) \% \text{capacity}$ predicts. `count` returns to 0 after every dequeue in this alternating pattern, correctly reflecting that the buffer never holds more than one element at a time in this particular sequence.

Quiz — trace the wrap

Given a capacity-4 circular queue with current state `front = 3`, `back = 1`, `count = 2`, work out, before checking the answer, what the new values of `back` and `count` become after one more `enqueue` call.

Quiz — answer

The new element is written into slot index 1 (the current value of `back`). `back` then advances to $(1 + 1) \bmod 4 = 2$. `count` increases from 2 to 3. Note that `front` is untouched by an `enqueue` — only a `dequeue` moves `front` — confirming that the two markers move entirely independently of each other, each only in response to its own operation.

Summary

A circular buffer avoids the shifting cost of a naive array-backed queue by wrapping indices using modulo arithmetic instead of physically moving elements, keeping both `enqueue` and `dequeue` genuinely $O(1)$. The condition `front == back` is ambiguous between a fully empty and a fully full buffer, and the correct resolution is to track element `count` explicitly rather than attempting to infer it from the two markers alone. Compared to `collections.deque`, a circular buffer trades away dynamic growth for a fixed, pre-allocated memory footprint known exactly in advance — a guarantee real systems in audio processing, networking, and logging rely on directly. Next lecture leaves arrays behind entirely and introduces linked lists, which have no fixed capacity and a genuinely different memory model from everything covered so far.